

DEEP LEARNING FOR FINANCE

MODULE 4 | LESSON 4

Multivariate RNNs

Reading Time	45 minutes
Prior Knowledge	RNN, Deep Neural Network, Market timing
Keywords	Multivariate RNN, VIX, S&P500

1. Introduction: S&P500 index and the VIX

It is now time to augment our deep RNNs to consider more than one series of inputs. In this lesson we revisit the market timing example of Lesson 2, but we will now predict stock prices using past information (prices) about the security as well as a leading indicator variable. Specifically, we will try to predict the prices of an Exchange Traded Fund (ETF) that replicates the composition and behavior of the S&P500 index in the U.S., the famous 'SPY'. And we will use for that not only past prices of SPY, but also the prices (levels) of a leading indicator of S&P500 index: the VIX index.

The VIX is a volatility index of the S&P500 market index. It is constructed using the implied volatility from a portfolio of 30-day constant-maturity options on S&P500 index. Since we all well know at this point, the fact that VIX index is constructed using options-implied volatility information means that it reflects investor's expectations on the underlying asset, the S&P500 index (Ticker symbol 'SPX'). More information on the VIX, and how it can be useful in the next few bullet points (please note these materials are only informative, not investment recommendation!):

- More information about the VIX index by the 'creators', Chicago Board Options Exchange (CBOE): https://www.cboe.com/tradable_products/vix/faqs/
- How investors & traders use the VIX: <https://optionshawk.com/how-to-use-the-vix-as-a-market-signal/>
- A web were you can consult the 'term structure' of Future contracts over VIX with different maturities: <http://vixcentral.com/>
- How to interpret the VIX term structure: <https://www.projectfinance.com/vix-term-structure/>
- An interesting paper on how to use VIX term structure to predict recessions: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3943982

- A great academic paper on how VIX term structure predicts risk premia:
<https://www.cambridge.org/core/services/aop-cambridge-core/content/view/56572D1F060448571BD8F597C732D9C3/S0022109017000825a.pdf/r>
- On using VIX in investing under a formal, systematic approach: <https://joi.pm-research.com/content/23/3/130.short>

When you come to think about it, if VIX really works as a powerful leading indicator of SPX, adding it as an input to our RNN prediction problem should be like 'cheating'. If we think about another type of prediction task suited for RNNs, such as language models, it would be like asking the model to predict how a sentence would end knowing what are the next few words. It should simplify the prediction task a lot!

Spoiler alert! It will not. Or, at least, it will not necessarily do it. For example, what if investors already looked at VIX prices to decide on their trading in the S&P500? In that case, it is likely that adding the VIX to our input matrix set could be even redundant, as this information is already incorporated into prices themselves.

Of course, it may be the case that investors (or at least some of them) are oblivious to information contained in VIX. For starters, it is likely that investors trading the S&P500 index and its components are inherently different from those trading options on the SPX. These differences can arise not only in terms of how informed are each of these guys, but also their investment horizon, risk-aversion, ... All in all, these features (and others, like limits to arbitrage) cast doubts (at least ex-ante) on whether S&P500 incorporates data on VIX and the potential predictive power of VIX as a leading indicator.

So... let's resolve this question!

2. Data gathering

As usual, we will start with the data gathering process. Note how in this case we do not only gather daily stock prices from our S&P500 index ETF, SPY, from Yahoo! finance, but also on the daily prices (levels) of VIX. We will be working, as usual, with daily prices and trying to predict what is the next price that SPY will take.

```
In [1]: import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Read the CSV file
file_path = 'SPY.csv'
spy_df = pd.read_csv('SPY.csv', sep=';', decimal=',')
spy_df['Date'] = pd.to_datetime(spy_df['Date'], format='%m/%d/%y')
# Create a column for next day's SPY price (Output)
spy_df['Output'] = spy_df['Price'].shift(-1)

spy_df = spy_df.dropna()
```

```
spy_df.head()
```

Out[1]:

	Date	Price	CVol	% Change	Open	Low	High	NAV	Total Return (Gross)	% Return	Ci
1	2025-05-13	586.84	67947234	0.66	583.41	582.84	589.08	587.02	776.06	0.66	
2	2025-05-12	582.99	78993570	3.30	581.47	577.04	583.00	582.79	770.97	3.30	
3	2025-05-09	564.34	37603434	-0.13	566.48	562.76	567.50	564.37	746.30	-0.13	
4	2025-05-08	565.06	65130793	0.70	565.24	561.70	570.31	564.68	747.26	0.70	
5	2025-05-07	561.15	55733230	0.42	560.15	556.04	563.82	561.42	742.09	0.42	

```
In [2]: vix_df = pd.read_csv('VIX.csv', sep=';', decimal=',')
vix_df['Date'] = pd.to_datetime(vix_df['Date'], format='%m/%d/%y')
vix_df.head()
```

Out[2]:

	Date	Price	CVol	Change	% Change	Total Return (Gross, Unhedged)	Cumulative Return % (Gross, Unhedged)	Open	High	Low
0	2025-05-14	18.22	NaN	NaN	NaN	NaN	NaN	NaN	18.22	18.22
1	2025-05-13	18.22	NaN	-0.17	-0.92	NaN	NaN	18.69	18.85	17.65
2	2025-05-12	18.39	NaN	-3.51	-16.03	NaN	NaN	19.84	20.44	18.14
3	2025-05-09	21.90	NaN	-0.58	-2.58	NaN	NaN	22.40	22.82	21.83
4	2025-05-08	22.48	NaN	-1.07	-4.54	NaN	NaN	22.84	23.61	21.88

```
In [3]: # Merge the two dataframes on Date
df = pd.merge(
    spy_df[['Date', 'Price', 'Output']],
    vix_df[['Date', 'Price']],
    on='Date',
    how='inner'
)

# Rename columns for clarity
df = df.rename(columns={
    'Price_x': 'SPY_Price',
    'Price_y': 'VIX_Price'
})
```

```

# Drop any rows with NaN values (will be the last row due to shift operation)
df = df.dropna()

# Sort by date (newest first, matching the original data order)
df = df.sort_values('Date', ascending=False)

# Reset index
df = df.reset_index(drop=True)

# Reorder columns to make Output the last column
df = df[['Date', 'VIX_Price', 'SPY_Price', 'Output']]

# Display the first few rows of the final dataframe
df.head()

```

Out[3]:

	Date	VIX_Price	SPY_Price	Output
0	2025-05-13	18.22	586.84	582.99
1	2025-05-12	18.39	582.99	564.34
2	2025-05-09	21.90	564.34	565.06
3	2025-05-08	22.48	565.06	561.15
4	2025-05-07	23.55	561.15	558.80

3. Validation, train and test sample definition

Next, we proceed to define our training, test, and validation sets. Note that we will train using the 98% of the sample. The validation set will be 20% of the training test. We also need to define a 'window' of observations that we are going to take as a sequence input to our network. In this case we choose a window of 20, which means that the last 20 prices (observations) will be incorporated into the sequence of inputs for the network. In this case this means that we will have a sequence of 20 SPY prices, as well as 20 VIX prices that will be part of the training at each t .

```

In [5]: # First, we deal with the definition of the training, test, and validations sets
val_split = 0.2
train_split = 0.98
train_size = int(len(df) * train_split)
val_size = int(train_size * val_split)
test_size = int(len(df) - train_size)

window_size = 20 # How many daily prices do I want to incorporate in the input

ts = test_size
split_time = len(df) - ts
test_time = df.iloc[split_time + window_size :, 0:1].values

# Second, we actually define the input matrix for train (X_train) and test (X_test)
Xdf, ydf = (
    df.iloc[:, 1:3],
    df.iloc[:, -1],
) # Note how we choose close prices from both SPY and VIX in the X matrix

```

```

X = Xdf.astype("float32")
y = ydf.astype("float32")

y_train_set = y[:split_time]
y_test_set = y[split_time:]

X_train_set = X[:split_time]
X_test_set = X[split_time:]

n_features = X_train_set.shape[1]

# Third, we proceed with scaling inputs to the model.
scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train_set)
X_train_set_scaled = scaler_input.transform(X_train_set)
X_test_set_scaled = scaler_input.transform(X_test_set)

mean_ret = np.mean(y_train_set)

scaler_output = MinMaxScaler(feature_range=(-1, 1))
y_train_set = y_train_set.values.reshape(len(y_train_set), 1)
y_test_set = y_test_set.values.reshape(len(y_test_set), 1)
scaler_output.fit(y_train_set)
y_train_set_scaled = scaler_output.transform(y_train_set)

## Lastly, because we want a time series with up to 20 (window_size) past obser
training_time = df.iloc[:split_time, 0:1].values

X_train = []
y_train = []

for i in range(window_size, y_train_set_scaled.shape[0]):
    X_train.append(X_train_set_scaled[i - window_size : i, :])
    y_train.append(y_train_set_scaled[i])

X_train, y_train = np.array(X_train), np.array(y_train)

print(X_train.shape)
print(y_train.shape)

X_test = []
y_test = y_test_set

for i in range(window_size, y_test_set.shape[0]):
    X_test.append(X_test_set_scaled[i - window_size : i, :])

X_test, y_test = np.array(X_test), np.array(y_test)
print(X_test.shape)
print(y_test.shape)
print(n_features)

```

(3757, 20, 2)

(3757, 1)

(58, 20, 2)

(78, 1)

2

4. Defining the RNN model

Once we have pre-processed the input and output data of our network, let's build it!

Our network architecture will be identical to the one from Lesson 2. We consider a simple RNN 5 hidden layers (4 simple RNN layers + 1 final dense layer). Each of the RNN layers will have 50 neurons. We apply a Dropout of 20% after each of the RNN layers. Finally, before the single-unit final output layer, we will apply a dense layer with 10 neurons.

As we already mentioned, this choice of architecture is as good as random. We leave to you to play around with the architecture and check how predictions vary.

Remember that a good resource to have nearby when implementing any Deep Neural Network is the TensorFlow/Keras documentation:

<https://www.tensorflow.org/guide/keras/rnn?hl=en>

```
In [6]: import tensorflow as tf

model = tf.keras.models.Sequential(
    [
        tf.keras.layers.SimpleRNN(
            50, return_sequences=True, input_shape=(X_train.shape[1], n_features
        ),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=True),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=True),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=False),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.Dense(10),
        tf.keras.layers.Dense(1),
    ]
)
```

2025-05-14 11:13:41.036553: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-05-14 11:13:41.077864: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2025-05-14 11:13:42.152523: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

/usr/local/lib/python3.11/site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Let's now check how our model looks like:

```
In [7]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 20, 50)	2,650
dropout (Dropout)	(None, 20, 50)	0
simple_rnn_1 (SimpleRNN)	(None, 20, 50)	5,050
dropout_1 (Dropout)	(None, 20, 50)	0
simple_rnn_2 (SimpleRNN)	(None, 20, 50)	5,050
dropout_2 (Dropout)	(None, 20, 50)	0
simple_rnn_3 (SimpleRNN)	(None, 50)	5,050
dropout_3 (Dropout)	(None, 50)	0
dense (Dense)	(None, 10)	510
dense_1 (Dense)	(None, 1)	11



Total params: 18,321 (71.57 KB)

Trainable params: 18,321 (71.57 KB)

Non-trainable params: 0 (0.00 B)

Now, the number of parameters in the first layer of the model changes a little bit. Where does the 2,650 come from?

Well, 50×50 (neurons in the layer) + 50×2 (neurons times number of model features) + 50×1 (bias terms) = 2,650

Note how the total model parameters is now 18,321. Only 50 more (because there is 1 more input feature) than the similar model from Lesson 2. Finally, note how the only impact on model parameters from incorporating 1 more feature comes in the first layer of the model. After that, there is no increase in the trainable parameters resulting from considering more input features. This is also important to take into account when considering how the 'size' of the model may impact the performance of backprop and gradient descent algorithm: It is much more about the 'length' of the input sequence than the number of input features themselves!

5. Model training

Now, we can proceed with model training.

```
In [8]: from keras.callbacks import EarlyStopping

hp_lr = 1e-4
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=hp_lr), loss="mean_absolute
)

es = EarlyStopping(
    monitor="val_loss", mode="min", verbose=1, patience=10, restore_best_weights
)

# fit the models
model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=500,
    batch_size=64,
    verbose=1,
    callbacks=[es],
)
```

```

Epoch 1/500
47/47 ————— 4s 21ms/step - loss: 0.6665 - val_loss: 0.3481
Epoch 2/500
47/47 ————— 1s 13ms/step - loss: 0.5605 - val_loss: 0.1852
Epoch 3/500
47/47 ————— 1s 12ms/step - loss: 0.4981 - val_loss: 0.1422
Epoch 4/500
47/47 ————— 1s 13ms/step - loss: 0.4719 - val_loss: 0.1242
Epoch 5/500
47/47 ————— 1s 13ms/step - loss: 0.4521 - val_loss: 0.2171
Epoch 6/500
47/47 ————— 1s 13ms/step - loss: 0.4232 - val_loss: 0.1209
Epoch 7/500
47/47 ————— 1s 12ms/step - loss: 0.3865 - val_loss: 0.0979
Epoch 8/500
47/47 ————— 1s 13ms/step - loss: 0.3772 - val_loss: 0.1061
Epoch 9/500
47/47 ————— 1s 12ms/step - loss: 0.3500 - val_loss: 0.0619
Epoch 10/500
47/47 ————— 1s 14ms/step - loss: 0.3317 - val_loss: 0.1167
Epoch 11/500
47/47 ————— 1s 14ms/step - loss: 0.3162 - val_loss: 0.0567
Epoch 12/500
47/47 ————— 1s 12ms/step - loss: 0.3087 - val_loss: 0.1183
Epoch 13/500
47/47 ————— 1s 12ms/step - loss: 0.2763 - val_loss: 0.1875
Epoch 14/500
47/47 ————— 1s 12ms/step - loss: 0.2662 - val_loss: 0.0869
Epoch 15/500
47/47 ————— 1s 12ms/step - loss: 0.2599 - val_loss: 0.1384
Epoch 16/500
47/47 ————— 1s 13ms/step - loss: 0.2407 - val_loss: 0.1587
Epoch 17/500
47/47 ————— 1s 14ms/step - loss: 0.2301 - val_loss: 0.0672
Epoch 18/500
47/47 ————— 1s 12ms/step - loss: 0.2247 - val_loss: 0.0813
Epoch 19/500
47/47 ————— 1s 12ms/step - loss: 0.2118 - val_loss: 0.1060
Epoch 20/500
47/47 ————— 1s 12ms/step - loss: 0.2041 - val_loss: 0.0615
Epoch 21/500
47/47 ————— 1s 13ms/step - loss: 0.1937 - val_loss: 0.1089
Epoch 21: early stopping
Restoring model weights from the end of the best epoch: 11.

```

```
Out[8]: <keras.src.callbacks.history.History at 0x703649975850>
```

Then we can make our predictions on the test sample based on the trained model parameters:

```

In [9]: prediction = model.predict(X_test)
        prediction = scaler_output.inverse_transform(prediction)
        prediction = prediction.flatten()

        values = np.array(y_test[window_size:])
        values = values.flatten()

```

```
2/2 ————— 1s 259ms/step
```

```
In [10]: df_predictions = pd.DataFrame(
    {"Date": test_time.flatten(), "Pred RNN": prediction, "values": values}
)
df_predictions.Date = pd.to_datetime(df_predictions.Date, format="%Y-%m-%d")
df_predictions.head()
```

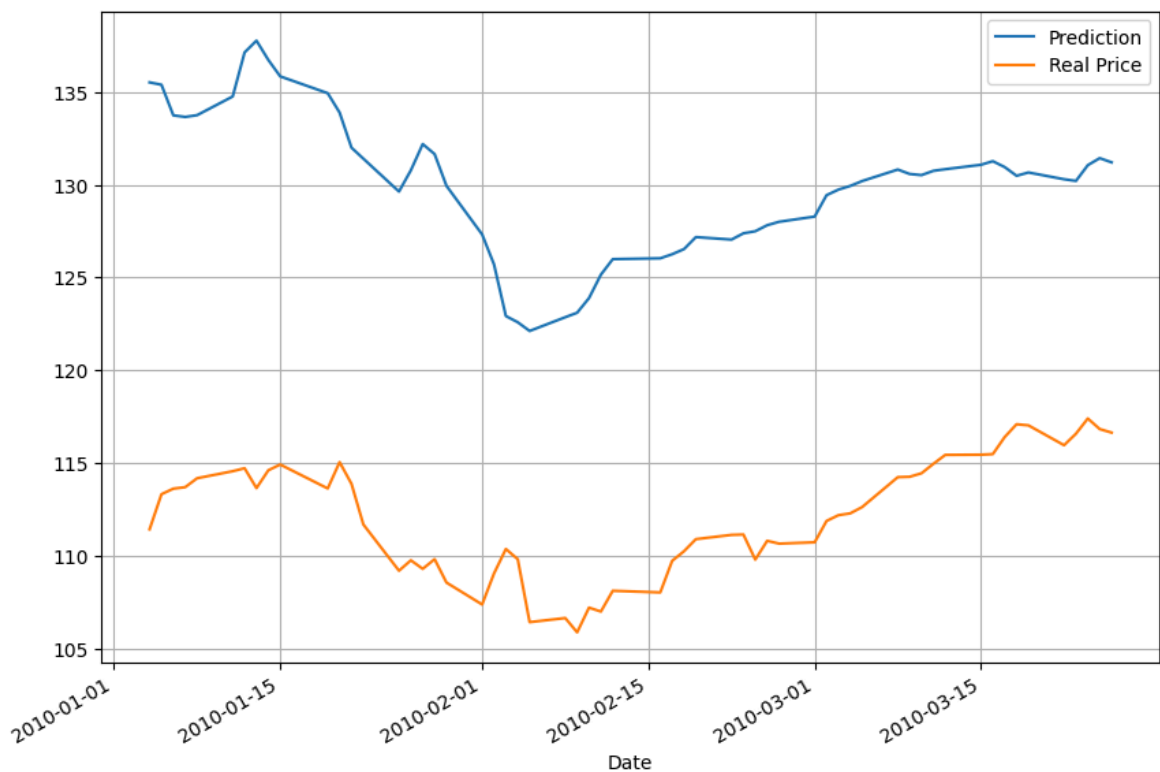
```
Out[10]:
```

	Date	Pred RNN	values
0	2010-03-26	131.208176	116.650002
1	2010-03-25	131.434296	116.839996
2	2010-03-24	131.056564	117.410004
3	2010-03-23	130.207977	116.589996
4	2010-03-22	130.297607	115.970001

And visualize it,

```
In [11]: import matplotlib.pyplot as plt

plt_1 = plt.figure(figsize=(10, 7))
ax = plt.gca()
df_predictions.plot(x="Date", y="Pred RNN", label="Prediction", ax=ax)
df_predictions.plot(x="Date", y="values", label="Real Price", ax=ax)
plt.grid()
plt.show()
```



As you can see (and please remember that we have not fixed the random seed so you may obtain different outcomes every time you train the model), the VIX does not really improve the predictive performance of the model by a lot (sometimes it's even making it worse!).

Can you think of ways to augment and improve this model? Well, one option may be using other types of 'leading variables'. For example, some variables used in practice for this are: prices of oil futures, manufacturing PMI in the U.S., agricultural production, ...

But we would usually not like a model that uses too many inputs because it will probably lead not only to increase training time and computational resources, but also to a lower degree of understanding regarding the interplay between the different features. Bottom line, it is important to understand the basics of how models work before we start adding inputs!

6. A final note on NLP

One of the main uses of RNNs is Natural Language Processing (NLP), which is a fascinating area. Sadly, there are, yet, not so many applications of NLP to financial market problems, which is why we have not focused on language models in this module.

In the book by Zhang. et. al. (2022) that we have been using as a reference for the whole course you have much more information on NLP models in case you are interested. Specifically, Chapters 15 and 16 deal with the pre-training and direct applications of NLP models in practice. Note that RNNs are just one part of NLP models, but probably the ones that make more sense when focusing on areas such as speech recognition.

Here you can have more info for the use of NLP in finance:

- A few examples of finance-related applications of NLP:
<https://softengi.com/blog/nlp-use-cases-in-finance/>
- One important application of NLP in finance is sentiment analysis. Here you have a quick review of it: <https://research.aimultiple.com/sentiment-analysis-stock-market/>
- A paper by Mittal and Goel (Stanford) that uses Twitter sentiment analysis for stock price prediction: <http://cs229.stanford.edu/proj2011/GoelMittal-StockMarketPredictionUsingTwitterSentimentAnalysis.pdf>

These are just some examples to 'whet your appetite'. Surely you can find many more!

7. Conclusion

In this lesson, we have dealt with multivariate RNNs. This means that we have used several input features into a recursive neural network model. So far, we have not obtained great results, though. But we have used a very tiny portion of all the available tools in deep learning and recursive networks.

In the next module, we will explore more specific types of RNN models: Gated Recurrent Units (GRUs) and Long Short-Term Memory (LSTM). See you in the next module!
